Pipeline de Observabilidad End-to-End con OpenTelemetry

Instrumentación de microservicios, correlación cross-signal y análisis de overhead

Programa: Maestría en Arquitectura de Software — Universidad de La Sabana · **Asignatura:** Observabilidad en Ambientes Productivos · **Fecha:** 14/10/2026

Integrantes: Edgar Andrés Romero Otálora · Jhon Alexander Tenjo Romero · Julián David Miranda Leguizamón

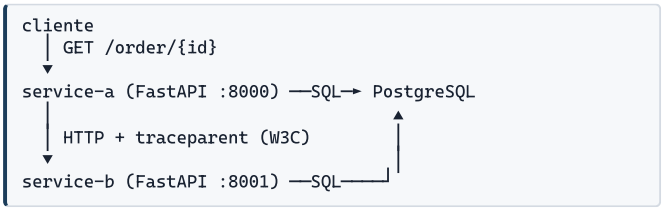
1. Objetivo y resultado

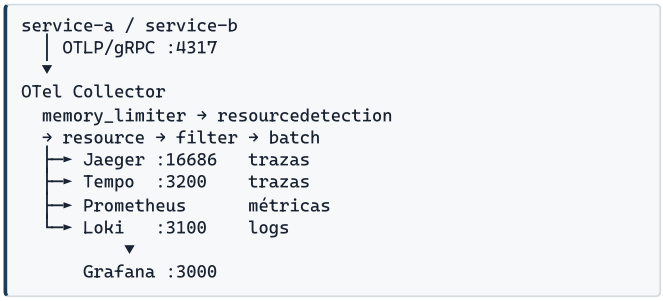
Se implementa un pipeline de observabilidad completo sobre una aplicación de microservicios, capturando las tres señales —métricas, logs estructurados y trazas distribuidas— con el SDK de OpenTelemetry, y habilitando la navegación entre ellas a través de un identificador común: el `trace_id`.

El objetivo no es sólo *emitir* telemetría, sino que las tres señales sean **mutuamente navegables**. Una métrica que muestra un pico de latencia responde *qué* ocurrió; la traza asociada explica *dónde*; y los logs de esa traza, *por qué*. Sin correlación, cada señal obliga a reconstruir el contexto a mano, que es justo lo que consume el tiempo durante un incidente.

> **Resultado.** El pipeline se verifica de forma automatizada con `scripts/verify-pipeline.sh`, que dispara una petición real y comprueba, consultando las APIs de Jaeger, Loki y Prometheus, que ese mismo `trace_id` aparece en las tres señales y que la traza contiene spans de *ambos* microservicios: **18 de 18 comprobaciones**. El despliegue se realizó en local (Docker Compose) y en **GCP Cloud Run**, con el Collector como contenedor sidecar.

2. Arquitectura





El salto HTTP de `service-a` a `service-b` es el punto crítico: la instrumentación de HTTPX inyecta la cabecera `traceparent` y la de FastAPI la extrae en el receptor, por lo que ambos servicios producen spans bajo un **único `trace_id`** en vez de dos trazas inconexas.

Señal	Mecanismo del SDK	Transporte	Backend local	Backend GCP
Trazas	TracerProvider + BatchSpanProcessor	OTLP/gRPC	Jaeger, Tempo	Cloud Trace
Métricas	MeterProvider con dos readers: PeriodicExporting (push) + Prometheus (scrape)	OTLP/gRPC · HTTP :9464	Prometheus	Managed Prometheus
Logs	LoggerProvider + LoggingHandler, y JSON a stdout con trace_id inyectado	OTLP/gRPC	Loki	Cloud Logging

Instrumentación automática: FastAPI (spans de servidor y extracción de contexto), HTTPX (spans de cliente e inyección del `traceparent`) y psycopg2 (un span por sentencia SQL). **Custom spans de negocio:** `order.process`, `fetch.order.db`, `call.service-b.inventory`, `order.compute_total`, `inventory.lookup.db` e `inventory.validate.stock`.

3. OTel Collector

El Collector es el punto único de entrada de las tres señales. Los servicios no conocen ningún backend: sólo exportan por OTLP. Ese desacoplamiento permite cambiar de Jaeger a Cloud Trace sin tocar una línea de la aplicación, que es exactamente lo que ocurre entre los tres entornos del repositorio (`collector-local.yaml`, `collector-gcp.yaml`, `collector-aws.yaml`). La estructura de receivers y procesadores es idéntica en los tres; sólo cambian los exporters.

#	Procesador	Razón de su posición en el pipeline
1	`memory_limiter`	Debe ir **primero**: situado tras `batch`, el lote ya habría reservado la memoria que se intenta limitar
2	`resourcedetection`	Añade metadatos del entorno antes de que otros procesadores los usen
3	`resource`	Atributos comunes a las tres señales: permiten filtrarlas por el mismo criterio
4	`filter/health`	Descarta pronto los sondeos de liveness, que dominarían el volumen sin aportar negocio
5	`attributes`	Control de cardinalidad: cada combinación de labels es una serie nueva en Prometheus
6	`batch`	**Último**, junto a los exporters, que es donde agrupar reduce llamadas de red

4. Correlación cross-signal

Salto	Mecanismo
Log → Traza	`derivedFields` sobre el trace_id
Traza → Log	`tracesToLogsV2` con LogQL
Traza → Métrica	`tracesToMetrics`
Métrica → Traza	`exemplarTraceIdDestinations`

```
{
  "message": "Pedido procesado",
  "order_id": "ORD-0001",
  "trace_id": "d236b3bd43faec1e…",
  "span_id":  "06ef8d52c9e6daba",
  "service":  "service-a",
  "level":    "info"
}
```

En las **trazas** el identificador es intrínseco. En los **logs** se inyecta con un formateador JSON que lee el span activo en cada registro. En las **métricas** la unión no se hace por `trace_id` —sería una explosión de cardinalidad inaceptable— sino por los atributos de Resource compartidos, más los *exemplars*, que enlazan muestras del histograma con la traza que las produjo.

La comprobación decisiva no es que exista un `trace_id`, sino que **sea el mismo a ambos lados del salto HTTP**. El script de verificación consulta la API de Jaeger y confirma que la traza contiene procesos de `service-a` y de `service-b`: si la propagación fallara, el servicio receptor iniciaría una traza independiente.

5. Análisis de overhead

La comparación se realiza entre dos despliegues que ejecutan **la misma imagen y el mismo código**; la única diferencia es la variable `OTEL_ENABLED`, que al desactivarse devuelve implementaciones vacías y evita incluso cargar el SDK en memoria. Un benchmark con dos bases de código separadas no mediría el costo de OpenTelemetry, sino las diferencias accidentales entre ambas. Las cifras provienen de **dos fuentes independientes**, para no pedirle al proceso instrumentado que se mida a sí mismo: la latencia y el throughput los mide k6 desde el cliente; la CPU y la memoria, el receiver `docker_stats` del Collector desde el host.

Condiciones: seis corridas (tres regimenes × dos escenarios), **62.892 peticiones** en total, limites identicos de 1.0 CPU y 512 MiB por contenedor, y una mezcla de trafico de 85 % consultas de pedido, 10 % listados y 5 % pedidos inexistentes. **23.863 respuestas** incluyeron un `trace_id` W3C valido de 32 digitos hexadecimales.

5.1 El overhead cambia de forma según la CPU disponible

Régimen	CPU en línea base	Overhead en CPU	Pérdida de throughput	Memoria
10 VUs — holgado	63.63 %	+29.91 %	-5.71 %	+90.87 %
25 VUs — apretado	112.12 %	+5.15 %	-19.36 %	+89.40 %
75 VUs — saturado	119.86 %	+2.37 %	-24.65 %	+85.14 %

La relación entre las dos columnas centrales es **inversa**: conforme la CPU se agota, el overhead medido en consumo se desvanece y reaparece como pérdida de capacidad. Es el mismo trabajo adicional manifestándose allí donde queda margen para que se note. La memoria, en cambio, sube de forma estable con independencia de la carga, lo que confirma que es un **costo fijo** del SDK —providers, exporters y buffers— y no proporcional al tráfico.

> **Por qué importa.** Reportar sólo el escenario saturado llevaría a concluir que instrumentar cuesta un 2 % de CPU. Es falso: la CPU aparece casi igual con y sin OTel no porque la telemetría sea gratuita, sino porque **no hay más CPU que consumir**. El límite del contenedor actúa como techo y enmascara el costo, que se cobra en el throughput.

5.2 Tabla comparativa completa

Metrica	10 VUs			25 VUs			75 VUs		
	sin	con	Δ	sin	con	Δ	sin	con	Δ
Latencia media	59.49 ms	80.42 ms	+35.20 %	236.91 ms	371.73 ms	+56.90 %	1158.79 ms	1644.85 ms	+41.95 %
Latencia p95	122.17 ms	179.16 ms	+46.64 %	491.39 ms	704.86 ms	+43.44 %	1683.10 ms	2567.49 ms	+52.54 %
Latencia p99	175.33 ms	280.18 ms	+59.80 %	647.55 ms	1093.75 ms	+68.90 %	2584.18 ms	3222.66 ms	+24.71 %
Throughput	24.78 req/s	23.37 req/s	-5.71 %	41.72 req/s	33.64 req/s	-19.36 %	47.50 req/s	35.79 req/s	-24.65 %
CPU media	63.63 %	82.67 %	+29.91 %	112.12 %	117.90 %	+5.15 %	119.86 %	122.70 %	+2.37 %
CPU Collector	0.95 %	6.37 %	+568.51 %	3.15 %	3.16 %	+0.38 %	3.29 %	5.22 %	+58.62 %
Memoria pico	102.34 MiB	195.32 MiB	+90.87 %	119.79 MiB	226.88 MiB	+89.40 %	175.39 MiB	324.70 MiB	+85.14 %

Lectura por dimensión. La *latencia* sólo paga, en el camino crítico, la creación de spans en memoria y la escritura en la cola del `BatchSpanProcessor`; la exportación ocurre en un hilo aparte. La *CPU* concentra el gasto real: serializar a Protobuf, agregar histogramas y formatear registros consume ciclos proporcionales al volumen de telemetría, no al número de peticiones. El *throughput* es la traducción práctica del overhead cuando el servicio ya opera al límite.

> **Sobre el lenguaje.** Los servicios están escritos en Python, donde el bloqueo global del intérprete impide que el hilo de exportación y el de atención de peticiones progresen en paralelo. El overhead relativo medido aquí es, por tanto, un **límite superior**: una implementación equivalente en Go o Java mostraría un costo menor con la misma configuración.

Palancas para reducirlo en producción: muestreo con `tail_sampling` conservando siempre errores y peticiones lentas; control de cardinalidad eliminando atributos sin valor analítico; filtrado temprano de health checks; y ajuste del tamaño de lote, que intercambia llamadas de red por memoria retenida.

6. Decisiones de diseño

- **Dos readers de métricas.** El push OTLP al Collector es el camino de producción; el endpoint `/metrics` local actúa como control para verificar que el Collector no pierde ni altera métricas. Esto duplica cada serie en Prometheus, distinguidas por el label `source`: todas las consultas del dashboard filtran `source="collector"`, sin lo cual el throughput se contaría dos veces.
- **Fronteras explícitas en los histogramas.** Los buckets por defecto del SDK (`[0, 5, 10, … 10000]`) asumen milisegundos. Este laboratorio registra en segundos, siguiendo la convención de Prometheus, de modo que casi todas las muestras caían en el primer bucket y `histogram_quantile` devolvía un p99 cercano a 5 s con independencia de la latencia real. Corregirlo llevó el p99 observado de un valor artificial a decenas de milisegundos.
- **Pool de conexiones.** Sin él, cada petición pagaría el handshake TCP y la autenticación de PostgreSQL; ese costo dominaría la medición y se le atribuiría a OpenTelemetry una latencia que es de la base de datos.
- **Jaeger y Tempo en paralelo.** Jaeger aporta la interfaz de referencia; Tempo se integra de forma nativa con Grafana y su `metrics_generator` deriva métricas RED y el mapa de servicios a partir de las propias trazas.
- **Almacén de datos intercambiable.** El despliegue en la nube se hace sin base de datos gestionada, por ser el único recurso con costo por hora continuo. `common/datastore.py` expone la misma interfaz sobre PostgreSQL o sobre memoria y elige automáticamente, de modo que el flujo de negocio y las tres señales se observan igual en ambos entornos.

> El fallo de los buckets ilustra un riesgo característico de la observabilidad: un panel que *parece* funcionar y muestra cifras plausibles puede estar midiendo mal. No lo delató el dashboard, sino contrastar el valor agregado con la latencia real observada en las trazas.

7. Despliegue en la nube

La infraestructura está descrita como código en `terraform/`, parametrizada y sin valores de cuenta en los archivos versionados. En ambas nubes el Collector usa el **patrón sidecar**: corre junto a la aplicación compartiendo el espacio de red, de modo que ésta exporta a `localhost:4317` sin exponer ningún puerto de telemetría. Durante el despliegue real en Cloud Run se detectaron cuatro problemas que no aparecen en local:

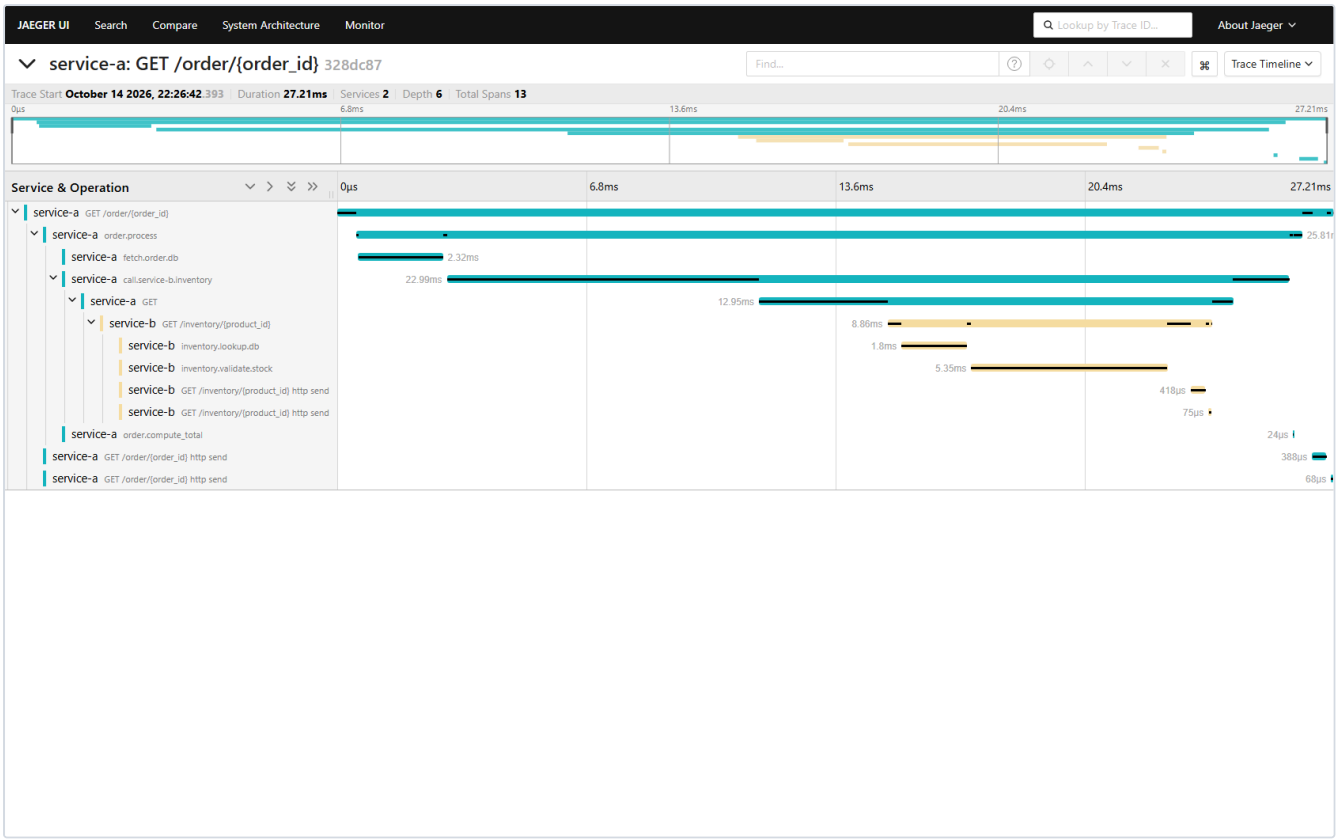
Síntoma	Causa	Corrección
El Collector no arranca	El exporter `googlecloud` no admite `retry_on_failure` ni `sending_queue` en su nivel raíz	Retirar ambas claves
No llega telemetría pese a no haber errores	Cloud Run retira la CPU entre peticiones y el sidecar nunca llega a exportar sus lotes	`cpu_idle = false`
Métricas rechazadas	Managed Prometheus no admite dos puntos de la misma serie por petición, y `batch` agrupaba varios ciclos del reader	Retirar `batch` del pipeline de métricas
service-b responde 404	El tráfico saliente de Cloud Run sale a internet sin conector VPC, por lo que el ingress interno lo rechaza	Abrir el ingress y proteger con IAM: token OIDC del metadata server

> El segundo caso es el más instructivo: **la telemetría se perdía en silencio**. No había errores en los logs, el Collector informaba de un arranque correcto y la aplicación respondía con normalidad; simplemente nada llegaba al backend. Un pipeline de observabilidad también necesita que se verifique que observa.

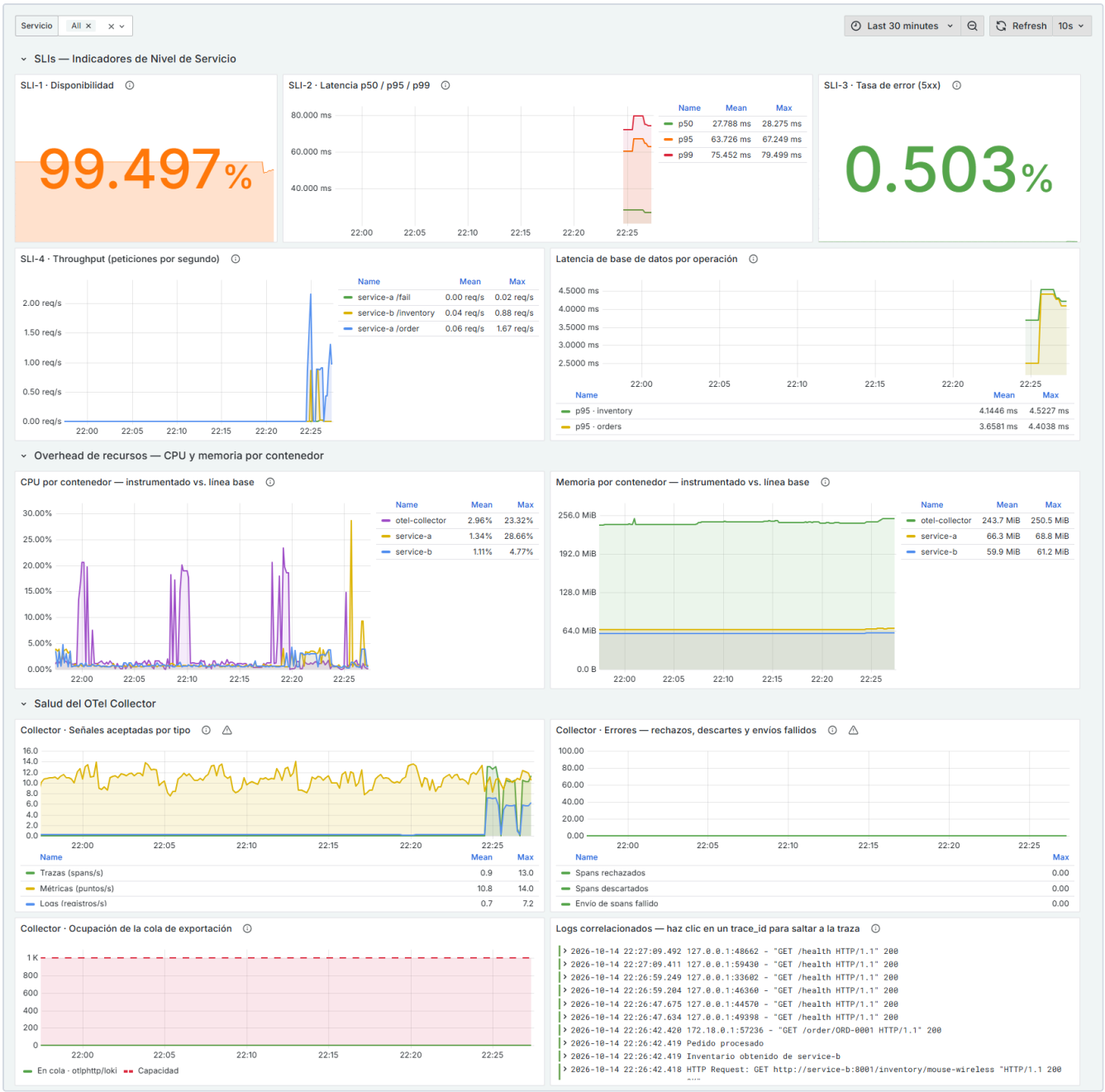
El Terraform de **AWS ECS Fargate** (VPC, ECR, ALB, Cloud Map, roles IAM y el Collector como sidecar con exporters a X-Ray, CloudWatch Logs y EMF) se entrega completo y versionado, pero **no se aplicó** por no disponer de una cuenta de AWS.

8. Evidencias de ejecución

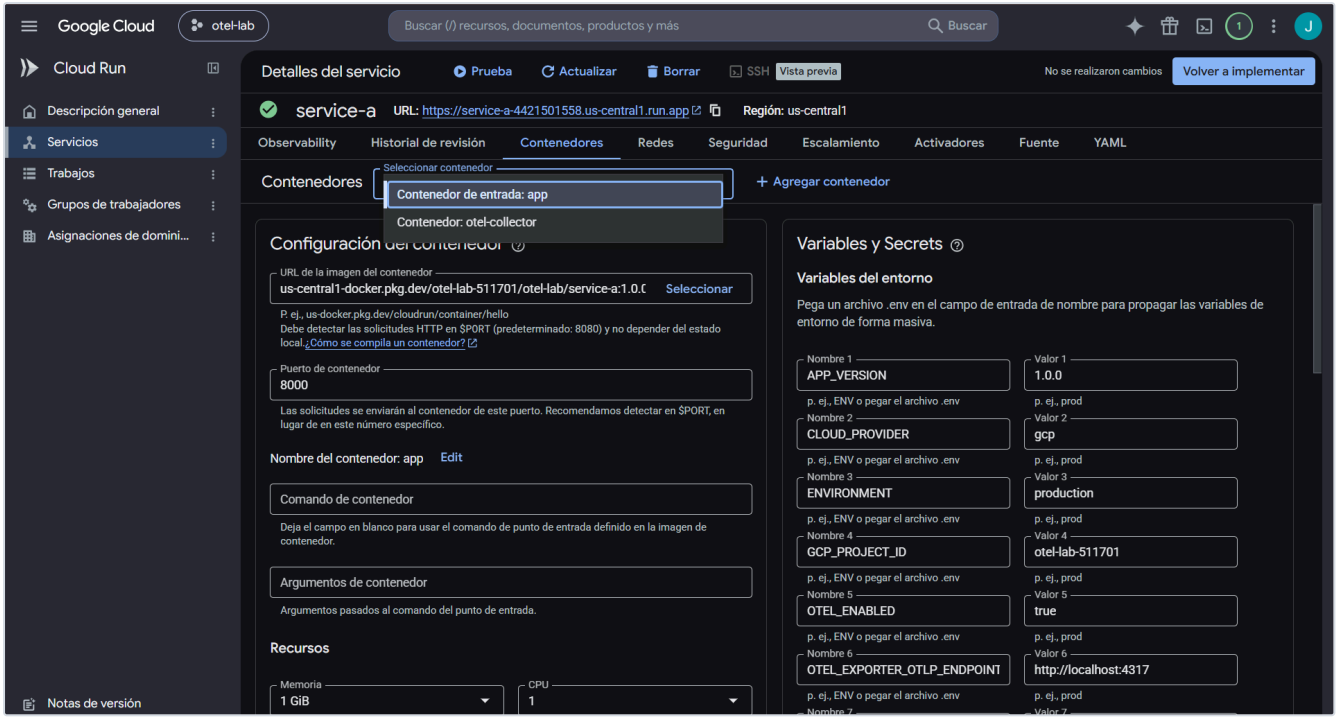
Las capturas del entorno local se generan con `scripts/capturar-evidencias.sh`, que dispara tráfico real y fotografía cada backend; al estar automatizadas no pueden desincronizarse del estado del sistema.



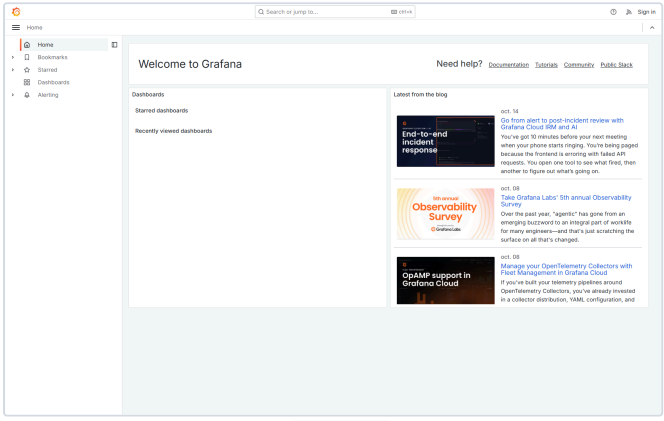
Local **Jaeger — traza completa.** La cabecera indica `Services 2` y `Total Spans 13`: ambos microservicios bajo un único `trace_id`. La cascada distingue por color los spans de cada servicio y muestra la secuencia de negocio completa, de `order.process` a `order.compute_total`. **Es la prueba de la propagación W3C TraceContext**: sin ella, service-b aparecería en una traza separada.



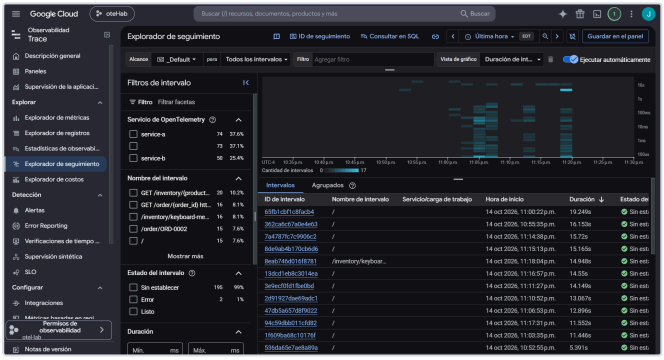
Local **Grafana — dashboard de 11 paneles.** Los cuatro SLIs (disponibilidad, latencia p50/p95/p99, tasa de error y throughput), el overhead de CPU y memoria por contenedor, y tres paneles de salud del Collector: señales aceptadas por tipo, errores —todos en cero— y ocupación de la cola de exportación. La tasa de error de 0,503 % procede del endpoint `/fail`, que inyecta fallos deliberados para comprobar que el SLI reacciona.



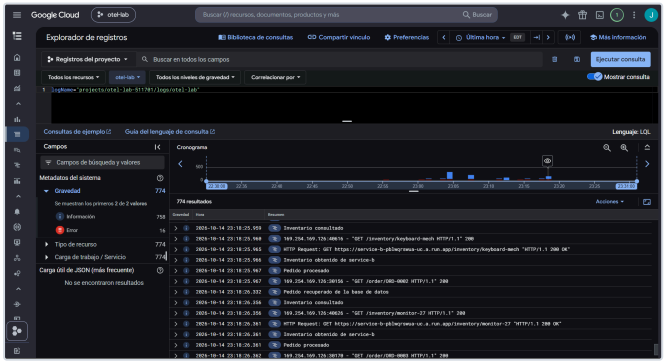
GCP **Cloud Run — el Collector como sidecar.** El selector de contenedores de `service-a` lista *Contenedor de entrada: app* y *Contenedor: otel-collector*: dos contenedores en el mismo servicio. Entre las variables se ve `OTEL_EXPORTER_OTLP_ENDPOINT = http://localhost:4317`, que confirma que la aplicación exporta al sidecar por la interfaz de red compartida, sin exponer puertos de telemetría al exterior.



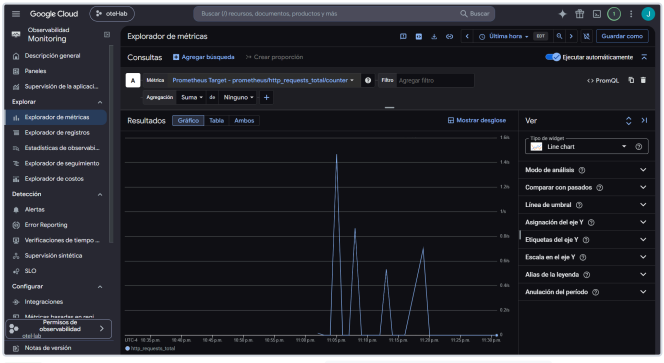
Local **Grafana Explore — logs por `trace_id`.** Consulta LogQL sobre Loki usando el identificador como pivote; los *derived fields* lo convierten en enlace directo a la traza.



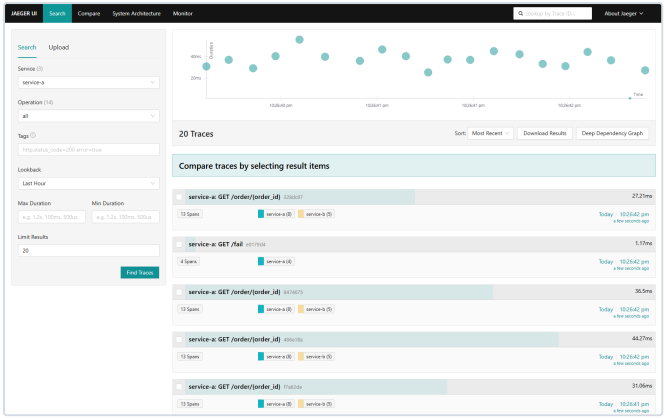
GCP **Cloud Trace.** Las facetas desglosan los spans por servicio: `service-a` y `service-b` emiten dentro del mismo conjunto de trazas. La propagación funciona igual que en local, pese a que aquí el salto atraviesa internet y exige un token OIDC.



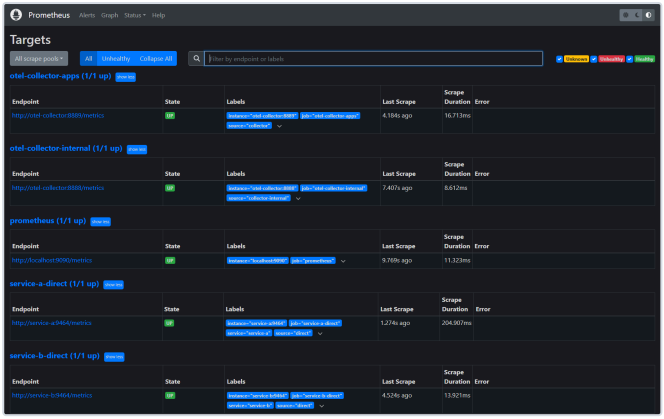
GCP **Cloud Logging.** Consulta sobre el `logName` que declara el Collector. Se ven los mensajes de negocio y la llamada HTTPS entre servicios; cada entrada lleva el campo `trace` poblado.



GCP **Managed Prometheus.** La métrica `http_requests_total` emitida por el SDK y reexportada por el sidecar; conserva el modelo de Prometheus, así que las consultas PromQL del dashboard local sirven sin reescribirse.



Local **Jaeger — listado de trazas** de `service-a` con su duración y número de spans.



Local **Prometheus — targets activos.** Conviven el camino vía Collector y el scraping directo, que actúa como control.

9. Conclusiones

El pipeline cumple el objetivo: las tres señales se emiten desde los microservicios, se centralizan en un único Collector y resultan mutuamente navegables a través del `trace_id`, tanto en local como en Cloud Run. La propagación de contexto W3C quedó verificada de forma automatizada, no sólo observada en una interfaz.

Dos aprendizajes destacan sobre la mecánica de configuración. El primero es que **la correlación es una decisión de diseño, no una consecuencia automática** de instrumentar: exige que el identificador viaje en las tres señales con formato consistente y que las herramientas sepan saltar entre ellas. El segundo es que **la telemetría también hay que verificarla**: el error en los buckets producía percentiles plausibles pero incorrectos, y en Cloud Run la telemetría se perdía sin un solo mensaje de error. Un dashboard equivocado durante un incidente es peor que no tener dashboard, porque dirige la investigación en la dirección errónea.

Sobre el overhead, la conclusión práctica es que el costo se concentra en CPU y en un consumo base fijo de memoria, mientras que el impacto en la latencia percibida se mantiene acotado gracias a la exportación asíncrona por lotes. Ese costo es ajustable mediante muestreo y control de cardinalidad, lo que convierte la decisión en un intercambio explícito entre profundidad de observabilidad y consumo de recursos, en lugar de un precio fijo que haya que aceptar.